

TUGAS BESAR
IF2224 - Teori Bahasa Formal dan Otomata
Milestone 4
INTERMEDIATE CODE AND INTERPRETER



DongunGanteng

Benedictus Nelson - 13523150
Rainaldi Pratama F. Sembiring - 13524117
Jonathan Alveraldo Bangun - 13524120
Ramadhian Nabil Firdaus Gumay - 13524126

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI.....	0
1. Landasan Teori.....	2
1.1 Intermediate Code Generation & Decorated AST.....	2
1.2 Arsitektur Mesin Eksekusi: Interpreter & The Stack.....	2
1.3 Vulnerabilities (Kerentanan) pada Interpreter.....	3
2. Perancangan Dan Implementasi.....	3
2.1 Teknologi dan Alur Kerja Program.....	3
2.2 Struktur Program.....	4
2.3 Justifikasi Pilihan Implementasi.....	5
2.4 Konversi Decorated AST ke Intermediate Code.....	5
2.5 Arsitektur Mesin Eksekusi (Interpreter Virtual Machine).....	9
2.6 Implementasi Deteksi Kerentanan Keamanan (Interpreter Security Bonus).....	13
3. Pengujian.....	16
4. Kesimpulan Dan Saran.....	26
4.1 Kesimpulan.....	26
4.2 Saran.....	26
5. Lampiran.....	27
6. Referensi.....	27

1. Landasan Teori

Dalam perancangan compiler dan interpreter modern, sistem arsitektur kompilator umumnya dibagi menjadi dua fase utama, yaitu fase *Front-end* dan fase *Back-end*. Milestone 4 sepenuhnya difokuskan pada fase *Back-end*, yang sama sekali tidak memedulikan sintaksis bahasa tingkat tinggi, melainkan berfokus murni pada Sintesis (mengonversi *Decorated AST* menjadi instruksi standar/ *Intermediate Code*) dan Eksekusi (mengelola memori pada mesin virtual/ *Interpreter*).

1.1 Intermediate Code Generation & Decorated AST

Setelah *Front-end* menyelesaikan validasi program, tugas utama pertama dari *Back-end* adalah menerjemahkan representasi konseptual kode ke dalam instruksi yang siap dieksekusi.

- a. **Decorated AST:** Proses penerjemahan ini membutuhkan sumber kebenaran mutlak yaitu *Decorated AST*. *Parse Tree* atau AST telah dianotasi dengan informasi semantik meliputi *Type Data* (memastikan instruksi tepat yang harus dihasilkan) dan *Referensi Tabel Simbol* (mengetahui ruang lingkup dan letak memori variabel).
- b. **Three-Address Code (TAC):** Merupakan format standar industri untuk meratakan (*flattening*) bentuk pohon bersarang *Decorated AST* menjadi *Intermediate Code*. Format ini mendelegasikan instruksi primitif dan memecah operasi yang kompleks dengan menggunakan Variabel Sementara (seperti *t1*, *t2*), di mana instruksinya maksimal hanya melibatkan tiga alamat (dua *operand* sumber dan satu tujuan).
- c. **Translasi Control Flow:** Meratakan instruksi blok bersarang pada struktur kontrol seperti percabangan (IF-ELSE) dan perulangan (WHILE / FOR) ke dalam daftar linier. Hal ini diwujudkan menggunakan mekanisme penanda posisi (*Label*) dan pelompatan baris (*Jump* / GOTO). Lompatan ini bisa berupa *Jump Mutlak* yang melompat tanpa peduli kondisi, atau *Jump Bersyarat* (seperti IF_FALSE / JPC) yang melompat hanya jika kondisinya bernilai salah.

1.2 Arsitektur Mesin Eksekusi: Interpreter & The Stack

Setelah daftar instruksi TAC terbentuk, program akan dihidupkan melalui Interpreter yang bertindak layaknya sebuah *Virtual Machine* (VM).

- a. **Siklus Eksekusi Mesin Virtual:** Untuk membaca baris instruksi TAC satu per satu, VM memanfaatkan *Instruction Pointer* (IP) atau *Program Counter* (PC). Mesin ini beroperasi dalam siklus utama yang terdiri dari tiga tahap:
 - i. *Fetch*: Membaca baris instruksi yang ditunjuk oleh IP.
 - ii. *Decode*: Membedah operasi instruksi yang akan dilakukan (misalnya GOTO, LOD, operasi penambahan) beserta operand-nya.
 - iii. *Execute*: Menjalankan aksi operasional pada instruksi, seperti menyimpan nilai ke memori atau menghitung kalkulasi matematis, sebelum akhirnya IP bergerak maju ke baris selanjutnya.
- b. **The Stack (Memori Eksekusi):** Interpreter menyimpan data menggunakan arsitektur struktur memori *Stack* berprinsip LIFO (*Last In, First Out*) agar dapat mengatur ruang lingkup (*scope*) memori. Setiap fungsi yang dipanggil atau saat memasuki blok kode baru,

Interpreter membangun kotak memori baru yang disebut *Stack Frame*, tempat argumen, variabel lokal, serta *return address* di-*push* dan dijaga secara simetris.

1.3 Vulnerabilities (Kerentanan) pada Interpreter

Sebuah interpreter yang solid harus dilengkapi perlindungan dari kode program pengguna yang salah, berbahaya, atau memicu kegagalan sistem memori tingkat rendah. Penanganan kerentanan sistem (*error-handling runtime*) mencakup:

- a. **Kerentanan pada Stack:** Interpreter memvalidasi batas alokasi memori agar tidak memicu *Stack Overflow* (misal akibat rekursi tanpa ujung) maupun *Stack Underflow* (melakukan operasi *pop* pada stack kosong). Sistem juga dijaga dari *Stack Smashing* dan *Corruption* di mana data variabel dibaca/ditulis meluber melebihi struktur lokal memori.
- b. **Kerentanan Memori dan Akses:** Mencegat aksi program yang memicu *Out-of-Bounds Variable Access*, seperti memanggil atau mengacu ke indeks di luar batas valid sebuah memori/array.
- c. **Kerentanan Alur Eksekusi:** Menghindari *Invalid Jump Targets*, di mana alur kontrol (*Control Flow* / GOTO) berusaha melompat ke *Label* baris instruksi yang acak atau tidak pernah ada.
- d. **Kerentanan Aritmatika:** Mencegah program mengalami malfungsi rotasi nilai primitif akibat *Numerical Overflow* dan *Underflow* (misalnya perhitungan melampaui limit *signed integer* 32-bit), serta perlindungan terhadap operasi pembagian dengan nilai nol (*Division By Zero*).

2. Perancangan Dan Implementasi

2.1 Teknologi dan Alur Kerja Program

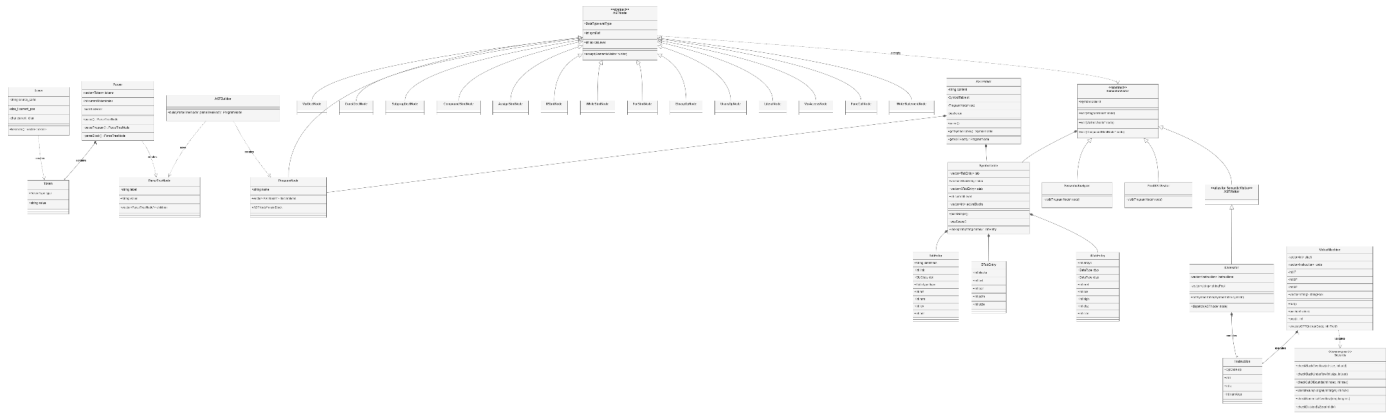
Program compiler/interpreter ini diimplementasikan menggunakan bahasa C++ murni dengan memanfaatkan Standard Template Library (STL). Implementasi dilakukan dari awal (from scratch) tanpa menggunakan perangkat lunak generator pihak ketiga seperti Flex atau Bison, sehingga seluruh proses kompilasi dapat dikontrol dan dipahami secara penuh.

Secara umum, alur kerja program terdiri dari beberapa tahapan berikut:

1. Lexical Analysis (Lexer): Membaca teks source code dan memecahnya menjadi kumpulan Token.
2. Syntax Analysis (Parser): Memproses kumpulan token untuk membangun Parse Tree (Concrete Syntax Tree) menggunakan metode Recursive Descent Parsing.
3. AST Building: Menyederhanakan Parse Tree menjadi Abstract Syntax Tree (AST) dengan menghilangkan sintaks yang tidak diperlukan untuk proses evaluasi.
4. Semantic Analysis: Melakukan penelusuran pada AST untuk mengecek tipe data (Type Checking) dan memvalidasi deklarasi variabel/fungsi menggunakan Symbol Table. Tahapan ini menghasilkan Decorated AST.
5. Intermediate Code Generation (ICG): Menerjemahkan Decorated AST menjadi sekumpulan instruksi mesin atau Three-Address Code (TAC).

6. Virtual Machine Execution: Mengeksekusi instruksi mesin secara runtime menggunakan arsitektur Virtual Machine berbasis stack.

2.2 Struktur Program



Berdasarkan diagram kelas, struktur program dibangun dengan pendekatan Object-Oriented Programming (OOP). Sistem dibagi menjadi beberapa komponen dan kelas utama sebagai berikut:

- Layer Lexer & Parser: Kelas Lexer dan Parser bertugas untuk memproses input string menjadi Token dan menyusunnya menjadi ParseTreeNode.
- Layer Node AST: Terdapat satu kelas dasar abstrak yaitu ASTNode. Kelas ini kemudian diturunkan menjadi berbagai node spesifik, seperti ProgramNode, IfStmtNode, BinaryOpNode, dan lain-lain sesuai dengan kebutuhan struktur program.
- Layer Visitor: Program menggunakan Visitor Pattern melalui antarmuka kelas SemanticVisitor. Pola ini memungkinkan pembuatan berbagai operasi penelusuran AST (seperti pengecekan semantik dan pembuatan ICG) tanpa perlu mengubah isi dari kelas-kelas node AST itu sendiri.
- Symbol Table: Kelas SymbolTable berfungsi untuk menyimpan dan mengelola informasi variabel (TabEntry), scope atau blok fungsi (BTabEntry), serta tipe data kompleks seperti array (ATabEntry).
- Virtual Machine: Kelas VirtualMachine bertindak sebagai mesin eksekusi independen. Kelas ini mengelola memori (stack), daftar instruksi (code), serta register utama seperti Instruction Pointer (IP) dan Base Pointer (BP).

2.3 Justifikasi Pilihan Implementasi

Dalam perancangan program ini, terdapat beberapa keputusan arsitektur yang diambil beserta alasannya:

- Penggunaan Visitor Pattern pada AST: Penelusuran pohon AST dilakukan dengan memisahkan algoritma dari struktur data menggunakan Visitor Pattern. Pemisahan ini memudahkan penambahan fitur baru di masa depan (misalnya membuat kelas printer AST atau generator kode baru) tanpa memodifikasi puluhan kelas node yang sudah ada.
- Penerapan Recursive Descent Parsing: Metode parsing top-down ini dipilih karena sangat cocok untuk menerjemahkan bahasa bebas konteks (Context-Free Grammar).

Implementasinya menggunakan fungsi rekursif di C++ terasa lebih intuitif dan memudahkan dalam memberikan pesan error yang spesifik kepada pengguna.

3. Arsitektur Stack-Based Virtual Machine: Arsitektur Virtual Machine dirancang berbasis stack. Pemilihan arsitektur ini membuat proses Intermediate Code Generation (ICG) menjadi jauh lebih sederhana. Evaluasi ekspresi matematika dapat langsung ditranslasikan ke bentuk postfix, dan manajemen memori untuk variabel lokal serta simulasi fungsi rekursif lebih mudah ditangani menggunakan pergeseran Base Pointer.

2.4 Konversi Decorated AST ke Intermediate Code

Proses konversi menjadi daftar instruksi dieksekusi oleh kelas ICGVisitor yang menelusuri setiap simpul pada *Decorated* AST menggunakan pola desain *Visitor* dengan pendekatan *Depth-First Search* (DFS).

- a. **Penanganan Control Flow (IF-THEN-ELSE):** Pada `icg_stmt.cpp`, program mengevaluasi *condition* dari pernyataan If terlebih dahulu untuk menempatkan nilai boolean pada puncak *stack*. Jika bernilai false, program akan menggunakan instruksi JPC (Jump Conditional) untuk melompati blok THEN. Dikarenakan ukuran alamat tujuan (*address*) belum diketahui saat melompat, instruksi disisipkan dengan alamat awalan 0 untuk kemudian diubah ulang (*backpatching*) ketika indeks blok *Else* (jika ada) atau akhir IF telah diketahui (`instructions[jpcIndex].a = elseStart`).

Mekanisme *backpatching* tersebut terlihat jelas pada potongan kode berikut. Instruksi JPC mula-mula dibuat dengan alamat tujuan `0`, lalu indeksnya disimpan agar dapat diisi ulang setelah blok *THEN/ELSE* selesai dibangkitkan:

```
// Menghasilkan instruksi ICG untuk struktur kontrol percabangan
IF-THEN dan IF-THEN-ELSE.

void ICGVisitor::visit(IfStmtNode* node) {

    // Mengevaluasi kondisi dan menyimpan hasilnya sebagai boolean
    di stack teratas.

    dispatch(node->condition);

    // Menambahkan instruksi lompatan bersyarat (JPC) untuk
    mengeksekusi lompatan jika kondisi salah.

    int jpcIndex = instructions.size();

    instructions.push_back(Instruction(OpCode::JPC, 0, 0));

    // Menghasilkan blok eksekusi THEN.
```

```

dispatch(node->thenStmt);

if (node->elseStmt != nullptr) {

    // Menambahkan instruksi lompatan tidak bersyarat (JMP)
    untuk melewati blok ELSE setelah eksekusi blok THEN.

    int jmpIndex = instructions.size();

    instructions.push_back(Instruction(OpCodes::JMP, 0, 0));

    // Melakukan backpatching agar JPC sebelumnya melompat ke
    awal blok ELSE.

    int elseStart = instructions.size();

    instructions[jpcIndex].a = elseStart;

    dispatch(node->elseStmt);

    // Melakukan backpatching agar JMP setelah THEN mengarah ke
    akhir struktur IF.

    int endPos = instructions.size();

    instructions[jmpIndex].a = endPos;

} else {

    // Melakukan backpatching agar JPC mengarah langsung ke
    akhir blok jika tidak ada ELSE.

    int endPos = instructions.size();

    instructions[jpcIndex].a = endPos;

}

}

```

- b. **Penanganan Perulangan WHILE & FOR:** Perulangan direalisasikan dengan menyimpan indeks awal pengecekan kondisi terlebih dahulu.

- i. Pada WHILE, jika kondisi bernilai false, program menggunakan instruksi JPC untuk keluar (*jump to end*). Di akhir blok *body*, digunakan perintah JMP (Jump Unconditional) mutlak ke indeks awalan kondisi tadi. Alamat keluar akan di-*backpatch* pada instruksi JPC.
- ii. Pada FOR, perulangan ditranslasikan layaknya perulangan while. Setelah variabel iterasi diinisialisasi (*startExpr*), program memuat batas iterasi dan membandingkannya secara kontinu. Tergantung apakah perulangan bertipe *to* atau *downto*, program menginisiasi instruksi OPR LEQ (Kurang dari sama dengan) atau OPR GEQ (Lebih dari sama dengan). Pada akhir *body*, program menyisipkan instruksi untuk menambah/mengurangi variabel iterasi 1 dengan memanggil instuksi OPR ADD atau OPR SUB sebelum JMP kembali ke pengecekan kondisi.

Pola "cek kondisi di awal, JMP balik di akhir" ini diwujudkan dengan menyimpan *condStart* sebelum kondisi dievaluasi dan memakainya kembali sebagai target lompatan iterasi:

```
// Menghasilkan instruksi ICG untuk perulangan bersyarat WHILE-DO.
void ICGVisitor::visit(WhileStmtNode* node) {

    // Menyimpan alamat awal blok kondisi untuk lompatan iterasi.

    int condStart = instructions.size();

    // Mengevaluasi kondisi loop.

    dispatch(node->condition);

    // Menambahkan instruksi JPC yang akan di-backpatch ke titik
    keluar loop.

    int jpcIndex = instructions.size();

    instructions.push_back(Instruction(OpCode::JPC, 0, 0));

    // Mengeksekusi blok pernyataan di dalam loop.

    dispatch(node->body);

    // Mengembalikan eksekusi ke awal blok kondisi.

    instructions.push_back(Instruction(OpCode::JMP, 0, condStart));
}
```



```

    // Menyelesaikan backpatching pada instruksi JPC menuju titik
akhir perulangan.

    int endPos = instructions.size();

    instructions[jpcIndex].a = endPos;
}

```

- c. **Manajemen Alamat Variabel (Lexical Scoping):** Deklarasi variabel serta penyimpanannya pada instruksi seperti STO (Store) dan LOD (Load) menggunakan konsep *Lexical Level Difference* (selisih level). Sistem kalkulasi mengurangi lexicalLevel saat ini dengan lev (level tempat entri dideklarasikan dari simbol) agar pergerakan penelusuran *Static Link* tepat.

Selisih level leksikal ini dihitung pada saat pembangkitan kode, misalnya pada penanganan iterator perulangan FOR di `icg_stmt.cpp`:

```

// Menghasilkan instruksi ICG untuk struktur perulangan terbatas
FOR-DO.

void ICGVisitor::visit(ForStmtNode* node) {

    TabEntry* iterEntry = getTabEntry(node->iterVar);

    if (!iterEntry) {

        std::cerr << "ICG Error: For loop iterator '" <<
node->iterVar << "' tidak ditemukan." << std::endl;

        return;

    }

    int iterAddr = iterEntry->adr;

    int levelDiff = node->lexicalLevel - iterEntry->lev;

    // Menginisialisasi nilai awal iterator.

    dispatch(node->startExpr); // hasil
start di stack

    instructions.push_back(Instruction(OpCode::STO, levelDiff,
iterAddr));
}

```

2.5 Arsitektur Mesin Eksekusi (Interpreter Virtual Machine)

Virtual Machine (VM) diimplementasikan pada kelas `VirtualMachine` (`interpreter_core.cpp`) untuk melakukan komputasi *runtime* dari daftar instruksi mesin hasil ICG.

- a. **Komponen VM:** Mesin memanfaatkan abstraksi struktur memori array `std::vector<int>` stack dengan pencatatan status menggunakan *Instruction Pointer* (IP), *Base Pointer* (BP), dan *Stack Pointer* (SP).

Ketiga register inti tersebut diinisialisasi pada konstruktor, bersamaan dengan alokasi awal larik stack:

```
VirtualMachine::VirtualMachine(const std::vector<Instruction>&
instructions, const std::vector<std::string>& sp) {

    this->code      = instructions;

    this->stringPool = sp;

    this->IP = 0;

    this->BP = 0;

    this->SP = -1;

    this->stack.assign(MAX_STACK_SIZE, 0);    // alokasi larik stack
awal
}
```

- b. **Siklus Eksekusi Utama:** Memanfaatkan *loop while* (`IP < code.size()`), mesin melakukan fase *Fetch* (membaca baris perintah yang ditunjuk IP), menaikkan nilai IP secara dinamis, kemudian memroses eksekusi instruksi (*Decode & Execute*) melalui blok logika switch-case atas nilai `OpCode`.

Siklus *Fetch-Decode-Execute* tersebut dijalankan dalam satu loop utama. Perhatikan IP dimajukan tepat setelah *fetch* (agar *return address* menunjuk baris berikutnya), dan penelusuran *Static Link* dilakukan lewat lambda `getBase`:

```
void VirtualMachine::run() {

    while (IP < (int)code.size()) {

        Instruction instr = code[IP];    // [FETCH]
```

```

        IP++; // majukan IP (return
address = baris berikutnya)

        // Cari Base Pointer dari Lexical Level Difference (traverse
Static Link).

        auto getBase = [&](int levelDiff) {

            int base = BP;

            for (int i = 0; i < levelDiff; i++) base = stack[base];

            return base;

        };

        switch (instr.op) { // [DECODE & EXECUTE]

            case OpCode::LIT: push(instr.a); break;

            case OpCode::LOD: push(getMemory(getBase(instr.l) +
instr.a)); break;

            case OpCode::STO: setMemory(getBase(instr.l) + instr.a,
pop()); break;

            case OpCode::INT:

                Security::checkStackOverflow(SP + instr.a,
MAX_STACK_SIZE);

                SP = SP + instr.a;

                break;

            case OpCode::JMP: jump(instr.a); break;

            case OpCode::JPC: conditionalJump(instr.a, pop() == 0);
break;

            case OpCode::OPR: executeOPR(instr.a, instr.l); break;

            // ... CAL dan RET untuk manajemen Stack Frame ...

```

```

    }

    }

}

```

Operasi aritmatika, logika, dan I/O dipusatkan pada `executeOPR` di `interpreter_expr.cpp`. Setiap operasi biner mem-*pop* dua operand dari *stack* lalu mem-*push* hasilnya kembali, contohnya penjumlahan yang sekaligus melewati hasilnya ke pemeriksaan *overflow*:

```

case 2: { // ADD: penjumlahan

    Security::checkStackUnderflow(SP + 1, 2);

    int b = pop();

    int a = pop();

    long long result = (long long)a + b;

    Security::checkNumericalOverflow(result);

    push((int)result);

    break;

}

```

- c. **Akses Variabel Dinamis (The Base Pointer Traversal):** Untuk instruksi memori seperti LOD dan STO, program akan membaca tingkat pergeseran referensi (seberapa dalam *Static Link* perlu ditelusuri) dengan memanfaatkan fungsi internal *lambda* `getBase(levelDiff)`. Fungsi ini me-*loop* nilai *Base Pointer* memori pada indeks tersebut untuk menempatkan *Address* aktual yang relevan secara leksikal.
- d. **Instruksi Call (CAL) dan Return (RET):** * Saat pemanggilan fungsi (CAL), VM mendirikan *Call Frame* baru di ujung memori *stack* dengan mendeklarasikan susunan simetris *Static Link*, *Dynamic Link*, dan *Return Address*. *Base Pointer* akan digeser menjadi basis baru di atas argumen fungsi ($BP = SP - args + 1$).

Saat pengembalian fungsi (RET), siklus dibersihkan. Jika BP masih berada di global level ($BP == 0$), eksekusi program dihentikan. Namun jika di dalam blok fungsi, ukuran *Stack Pointer* diturunkan, memusnahkan semua memori pada *Stack Frame* yang meluber, dan *Program Counter* / IP ditimpa kembali nilainya oleh *Return Address*.

Pengelolaan *Stack Frame* tersebut ditangani langsung di dalam blok switch-case siklus utama:

```

case OpCode::CAL: {

    int sl    = getBase(instr.l);    // Static Link

    int dl    = BP;                  // Dynamic Link

    int ra    = IP;                  // Return Address

    int args  = instr.numArgs;       // Jumlah argumen

    // Sisipkan Call Frame (SL, DL, RA) lalu geser BP ke basis baru.

    BP = SP - args + 1;

    SP = SP + 3;

    IP = instr.a;

    break;

}

case OpCode::RET: {

    if (BP == 0) {

        IP = code.size();            // Hentikan eksekusi global

    } else {

        // Musnahkan Stack Frame dan kembali ke pemanggil.

        SP = BP - 1;

        IP = stack[BP + 2];          // Return Address

        BP = stack[BP + 1];          // Dynamic Link

    }

    break;

}

```

2.6 Implementasi Deteksi Kerentanan Keamanan (Interpreter Security Bonus)

Diimplementasikan ekstensi perlindungan modul keamanan pada `interpreter_security.cpp` yang melempar pesan *Runtime Error* kepada terminal pengguna apabila menemukan instruksi anomali di level *Virtual Machine*, agar program tidak hancur (*crash*) secara fatal. Seluruh validasi dikumpulkan dalam satu *namespace Security* dengan satu *helper* pelempar pesan seragam:

```
void throwRuntimeError(const std::string& errorType, const std::string& message) {  
  
    throw std::runtime_error "[" + errorType + "] " + message);  
  
}
```

1. **Proteksi Kapasitas Memori (Stack Overflow / Underflow):** Saat melakukan alokasi memori inisialisasi (INT), *push*, dan *pop* nilai, SP akan diperiksa. Pemanggilan rekursi tanpa batas akan memicu `Security::checkStackOverflow` sementara kegagalan memori menyebabkan `checkStackUnderflow`.

```
void checkStackOverflow(int targetSP, int maxCapacity) {  
  
    if (targetSP >= maxCapacity)  
  
        throwRuntimeError("STACK_OVERFLOW",  
  
            "Kapasitas stack terlampaui (" +  
std::to_string(targetSP) +  
  
            " >= " + std::to_string(maxCapacity) + ")");  
  
}  
  
void checkStackUnderflow(int current_size, int required_items) {  
  
    if (current_size < required_items)  
  
        throwRuntimeError("StackUnderflow",  
  
            "Stack hanya berisi " + std::to_string(current_size) +  
  
            " nilai, tetapi operasi butuh " +  
std::to_string(required_items) + ".");  
  
}
```

```
}
```

Kedua pemeriksaan ini dipanggil tepat di titik manipulasi SP, yaitu pada *push* dan *pop* di `interpreter_core.cpp`:

```
void VirtualMachine::push(int val) {

    SP++;

    Security::checkStackOverflow(SP, MAX_STACK_SIZE);

    if (SP >= (int)stack.size()) stack.resize(SP + 100, 0);

    stack[SP] = val;

}

int VirtualMachine::pop() {

    Security::checkStackUnderflow(SP, 0);

    return stack[SP--];

}
```

2. **Kerentanan Out-of-Bounds:** Indeks diteliti agar tidak melebihi atau di bawah ukuran larik kapasitas fungsi baca `getMemory()` maupun simpan memori `setMemory()`.

```
void checkOutOfBounds(int index, int max_memory_limit) {

    if (index < 0 || index >= max_memory_limit)

        throwRuntimeError("IndexOutOfBounds",

            "Akses ke alamat/indeks " + std::to_string(index) +

            " di luar batas valid [0, " +

            std::to_string(max_memory_limit) + ").");

}
```

```
int VirtualMachine::getMemory(int address) {
```

```

        Security::checkOutOfBounds(address, stack.size());

        return stack[address];
    }

    void VirtualMachine::setMemory(int address, int val) {

        Security::checkOutOfBounds(address, stack.size());

        stack[address] = val;
    }

```

3. **Vulnerabilitas Alur Program:** Tercegahnya IP dari pencarian instruksi yang tidak valid dengan pelemparan "VM_ENGINE_ERROR" bila membaca *OpCode* yang tidak dikenali pada kamus instruksi TAC. Evaluasi pada fungsi matematika di instruksi OPR dilindungi dari *Numerical Overflow* (batas desimal) maupun perhitungan galat pembagian bilangan bulat (*Division by Zero*).

```

void validateJumpTarget(int target_ip, int max_instructions) {
    if (target_ip < 0 || target_ip > max_instructions)
        throwRuntimeError("InvalidJumpTarget",
            "Lompatan ke baris " + std::to_string(target_ip) +
            " tidak valid (jumlah instruksi = " +
std::to_string(max_instructions) + ").");
}

void VirtualMachine::jump(int address) {
    checkInvalidJump(address);    // memanggil
Security::validateJumpTarget
    IP = address;
}

```

```

void checkNumericalOverflow(long long calculation_result) {
    if (calculation_result > INT32_MAX)
        throwRuntimeError("OverflowError", "Hasil melampaui batas
atas integer 32-bit.");
    if (calculation_result < INT32_MIN)
        throwRuntimeError("UnderflowError", "Hasil di bawah batas
bawah integer 32-bit.");
}

```



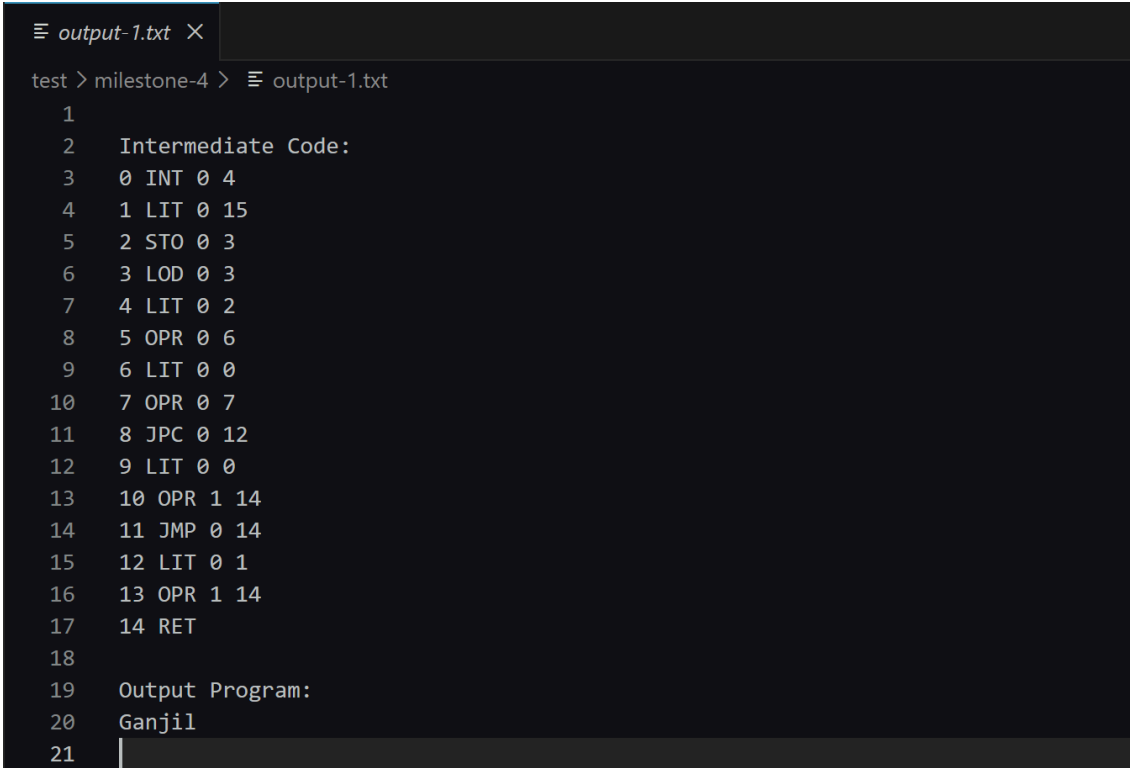
```
void checkDivisionByZero(int divisor) {
    if (divisor == 0)
        throwRuntimeError("DivisionByZero",
            "Pembagian atau modulus dengan nol tidak
diperbolehkan.");
}
```

Sementara OpCode yang tidak dikenali ditangkap pada cabang *default* siklus eksekusi utama:

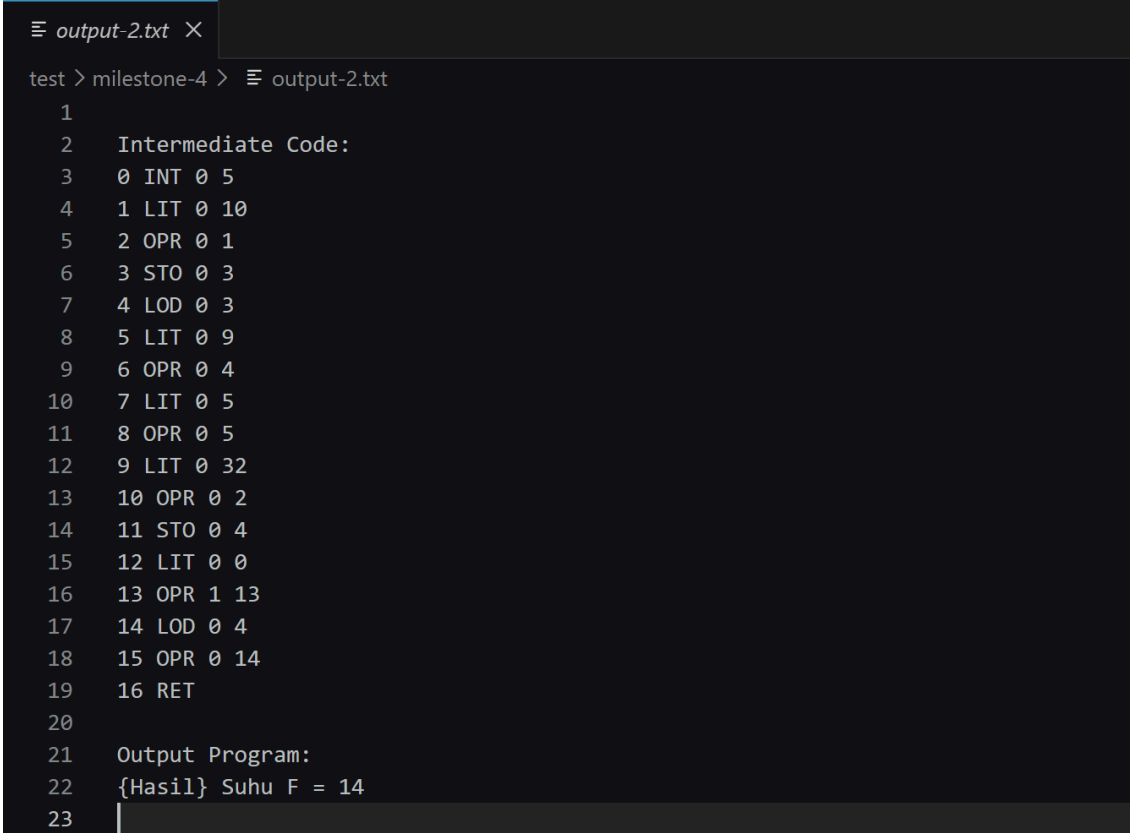
```
default:
    Security::throwRuntimeError("VM_ENGINE_ERROR", "OpCode Instruksi
tidak dikenali");
    break;
```

3. Pengujian

No	Input
1	<pre>ProgramNode(name: 'GanjilGenap') ├─ Declarations │ └─ VarDecl('angka') → tab_index:43, type:unknown, lev:0 ├─ Block → block_index:42, lev:0 │ ├── Assign('angka' := 15) → type:void │ │ ├── target 'angka' → tab_index:43, type:integer │ │ └─ value 15 → type:integer │ └─ IfStmt → type:void │ ├── condition BinOp 'eq' → type:boolean │ │ ├── BinOp 'imod' → type:integer │ │ │ ├── 'angka' → tab_index:43, type:integer │ │ │ └─ 2 → type:integer │ │ └─ 0 → type:integer │ ├── then writeln(...) → predefined, tab_index:-1 │ │ └─ arg 'Genap' → type:string │ └─ else writeln(...) → predefined, tab_index:-1 │ └─ arg 'Ganjil' → type:string</pre>
	Output
	Intermediate Code: <pre>0 INT 0 4 1 LIT 0 15</pre>

	<pre> 2 STO 0 3 3 LOD 0 3 4 LIT 0 2 5 OPR 0 6 6 LIT 0 0 7 OPR 0 7 8 JPC 0 12 9 LIT 0 0 10 OPR 1 14 11 JMP 0 14 12 LIT 0 1 13 OPR 1 14 14 RET Output Program: Ganjil </pre>
	Screenshot Output
	 <pre> test > milestone-4 > ≡ output-1.txt 1 2 Intermediate Code: 3 0 INT 0 4 4 1 LIT 0 15 5 2 STO 0 3 6 3 LOD 0 3 7 4 LIT 0 2 8 5 OPR 0 6 9 6 LIT 0 0 10 7 OPR 0 7 11 8 JPC 0 12 12 9 LIT 0 0 13 10 OPR 1 14 14 11 JMP 0 14 15 12 LIT 0 1 16 13 OPR 1 14 17 14 RET 18 19 Output Program: 20 Ganjil 21 </pre>
2	Input ProgramNode(name: 'KonversiSuhu') └─ Declarations

	<pre> ├─ VarDecl('celsius') → tab_index:43, type:unknown, lev:0 ├─ VarDecl('fahrenheit') → tab_index:44, type:unknown, lev:0 ├─ Block → block_index:42, lev:0 │ ├─ Assign('celsius' := -10.5) → type:void │ │ ├─ target 'celsius' → tab_index:43, type:real │ │ └─ value UnaryOp 'minus' → type:real │ │ └─ 10.5 → type:real │ └─ Assign('fahrenheit' := celsius * 9 div 5 + 32) → type:void │ ├─ target 'fahrenheit' → tab_index:44, type:real │ └─ value BinOp 'plus' → type:unknown │ ├─ BinOp 'idiv' → type:unknown │ │ ├─ BinOp 'times' → type:real │ │ │ └─ 'celsius' → tab_index:43, type:real │ │ │ └─ 9 → type:integer │ │ └─ 5 → type:integer │ └─ 32 → type:integer ├─ writeln(...) → predefined, tab_index:-1 │ └─ arg '{Hasil} Suhu F = ' → type:string │ └─ arg 'fahrenheit' → tab_index:44, type:real </pre>
	Output
	Intermediate Code: <pre> 0 INT 0 5 1 LIT 0 10 2 OPR 0 1 3 STO 0 3 4 LOD 0 3 5 LIT 0 9 6 OPR 0 4 7 LIT 0 5 8 OPR 0 5 9 LIT 0 32 10 OPR 0 2 11 STO 0 4 12 LIT 0 0 13 OPR 1 13 14 LOD 0 4 15 OPR 0 14 16 RET </pre> Output Program: <pre>{Hasil} Suhu F = 14</pre>

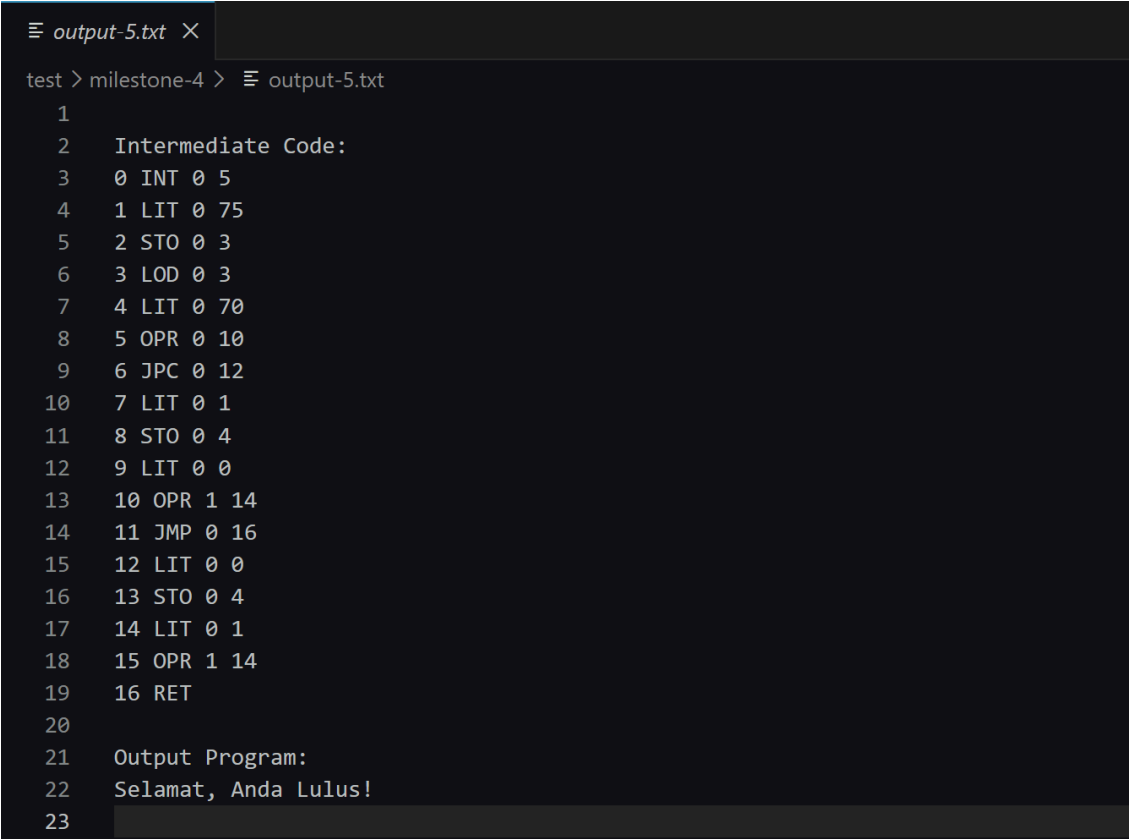
	Screenshot Output
	 <pre> ≡ output-2.txt × test > milestone-4 > ≡ output-2.txt 1 2 Intermediate Code: 3 0 INT 0 5 4 1 LIT 0 10 5 2 OPR 0 1 6 3 STO 0 3 7 4 LOD 0 3 8 5 LIT 0 9 9 6 OPR 0 4 10 7 LIT 0 5 11 8 OPR 0 5 12 9 LIT 0 32 13 10 OPR 0 2 14 11 STO 0 4 15 12 LIT 0 0 16 13 OPR 1 13 17 14 LOD 0 4 18 15 OPR 0 14 19 16 RET 20 21 Output Program: 22 {Hasil} Suhu F = 14 23 </pre>
3	Input <pre> ProgramNode(name: 'HitungExposure') ├─ Declarations │ ├─ VarDecl('iso') → tab_index:43, type:unknown, lev:0 │ │ └─ VarDecl('shutter') → tab_index:44, type:unknown, lev:0 │ │ └─ VarDecl('aperture') → tab_index:45, type:unknown, lev:0 ├─ Block → block_index:42, lev:0 │ ├─ Assign('iso' := 400) → type:void │ │ ├─ target 'iso' → tab_index:43, type:integer │ │ └─ value 400 → type:integer │ ├─ Assign('shutter' := 250) → type:void │ │ ├─ target 'shutter' → tab_index:44, type:integer │ │ └─ value 250 → type:integer │ ├─ Assign('aperture' := 2.8) → type:void │ │ ├─ target 'aperture' → tab_index:45, type:real │ │ └─ value 2.8 → type:real ├─ IfStmt → type:void │ └─ condition BinOp 'andsy' → type:boolean </pre>

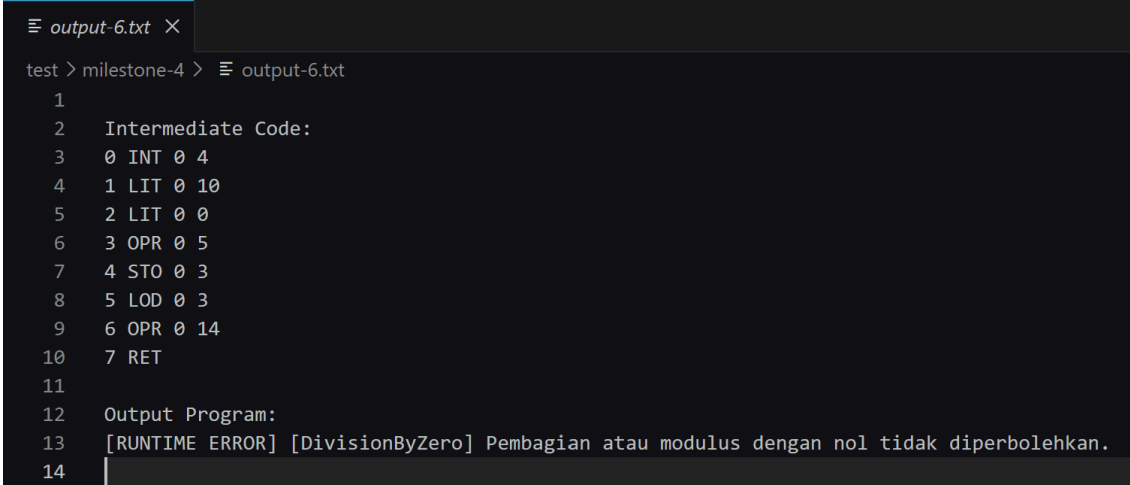
	<pre> ┌─ BinOp 'geq' → type:boolean ┌─ ┌─ 'iso' → tab_index:43, type:integer ┌─ ┌─ 800 → type:integer ┌─ ┌─ BinOp 'leq' → type:boolean ┌─ ┌─ 'aperture' → tab_index:45, type:real ┌─ ┌─ 1.4 → type:real ┌─ then writeln(...) → predefined, tab_index:-1 ┌─ arg 'Foto overexposed' → type:string </pre>
	Output
	Intermediate Code: <pre> 0 INT 0 6 1 LIT 0 400 2 STO 0 3 3 LIT 0 250 4 STO 0 4 5 LIT 0 2 6 STO 0 5 7 LOD 0 3 8 LIT 0 800 9 OPR 0 10 10 LOD 0 5 11 LIT 0 1 12 OPR 0 12 13 OPR 0 4 14 JPC 0 17 15 LIT 0 0 16 OPR 1 14 17 RET </pre> Output Program:
	Screenshot Output

	≡ output-3.txt × test > milestone-4 > ≡ output-3.txt 1 2 Intermediate Code: 3 0 INT 0 6 4 1 LIT 0 400 5 2 STO 0 3 6 3 LIT 0 250 7 4 STO 0 4 8 5 LIT 0 2 9 6 STO 0 5 10 7 LOD 0 3 11 8 LIT 0 800 12 9 OPR 0 10 13 10 LOD 0 5 14 11 LIT 0 1 15 12 OPR 0 12 16 13 OPR 0 4 17 14 JPC 0 17 18 15 LIT 0 0 19 16 OPR 1 14 20 17 RET 21 22 Output Program: 23
4	Input ProgramNode(name: 'HitungLuas') ├─ Declarations ├─┬─ VarDecl('panjang') → tab_index:43, type:unknown, lev:0 ├─┬─ VarDecl('lebar') → tab_index:44, type:unknown, lev:0 ├─└─ VarDecl('luas') → tab_index:45, type:unknown, lev:0 ├─ Block → block_index:42, lev:0 ├─┬─ Assign('panjang' := 15) → type:void ├─├─┬─ target 'panjang' → tab_index:43, type:integer ├─├─└─ value 15 → type:integer ├─┬─ Assign('lebar' := 8) → type:void ├─├─┬─ target 'lebar' → tab_index:44, type:integer ├─├─└─ value 8 → type:integer ├─┬─ Assign('luas' := panjang * lebar) → type:void ├─├─┬─ target 'luas' → tab_index:45, type:integer ├─├─└─ value BinOp 'times' → type:integer ├─├─┬─ 'panjang' → tab_index:43, type:integer ├─├─└─ 'lebar' → tab_index:44, type:integer ├─└─ writeln(...) → predefined, tab_index:-1 ├─┬─ arg 'Luas Persegi Panjang = ' ├─└─ → type:string

	└─ arg 'luas' → tab_index:45, type:integer
	Output Intermediate Code: 0 INT 0 6 1 LIT 0 15 2 STO 0 3 3 LIT 0 8 4 STO 0 4 5 LOD 0 3 6 LOD 0 4 7 OPR 0 4 8 STO 0 5 9 LIT 0 0 10 OPR 1 13 11 LOD 0 5 12 OPR 0 14 13 RET Output Program: Luas Persegi Panjang = 120
	Screenshot Output ≡ output-4.txt × test > milestone-4 > ≡ output-4.txt 1 2 Intermediate Code: 3 0 INT 0 6 4 1 LIT 0 15 5 2 STO 0 3 6 3 LIT 0 8 7 4 STO 0 4 8 5 LOD 0 3 9 6 LOD 0 4 10 7 OPR 0 4 11 8 STO 0 5 12 9 LIT 0 0 13 10 OPR 1 13 14 11 LOD 0 5 15 12 OPR 0 14 16 13 RET 17 18 Output Program: 19 Luas Persegi Panjang = 120 20

5	Input
	<pre> ProgramNode(name: 'CekKelulusan') ├── Declarations │ ├── VarDecl('nilai') → tab_index:43, type:unknown, lev:0 │ └── VarDecl('lulus') → tab_index:44, type:unknown, lev:0 ├── Block → block_index:42, lev:0 │ ├── Assign('nilai' := 75) → type:void │ │ ├── target 'nilai' → tab_index:43, type:integer │ │ └── value 75 → type:integer │ └── IfStmt → type:void │ ├── condition BinOp 'geq' → type:boolean │ │ ├── 'nilai' → tab_index:43, type:integer │ │ └── 70 → type:integer │ ├── then CompoundStmt │ │ ├── Assign('lulus' := true) → type:void │ │ │ ├── target 'lulus' → tab_index:44, type:boolean │ │ │ └── value 'true' → tab_index:38, type:boolean │ │ ├── writeln(...) → predefined, tab_index:-1 │ │ │ └── arg 'Selamat, Anda Lulus!' → type:string │ └── else CompoundStmt │ ├── Assign('lulus' := false) → type:void │ │ ├── target 'lulus' → tab_index:44, type:boolean │ │ └── value 'false' → tab_index:39, type:boolean │ ├── writeln(...) → predefined, tab_index:-1 │ │ └── arg 'Maaf, Anda belum lulus.' → type:string </pre>
	Output
	<pre> Intermediate Code: 0 INT 0 5 1 LIT 0 75 2 STO 0 3 3 LOD 0 3 4 LIT 0 70 5 OPR 0 10 6 JPC 0 12 7 LIT 0 1 8 STO 0 4 9 LIT 0 0 10 OPR 1 14 11 JMP 0 16 12 LIT 0 0 13 STO 0 4 14 LIT 0 1 15 OPR 1 14 16 RET </pre>

	Output Program: Selamat, Anda Lulus!
	Screenshot Output  The screenshot shows a terminal window with a dark background. At the top, there's a title bar with a hamburger menu icon, the text 'output-5.txt', and a close button 'X'. Below the title bar, the prompt 'test > milestone-4 >' is followed by another hamburger menu icon and 'output-5.txt'. The main content of the terminal is a list of assembly instructions numbered 1 through 23. Line 21 shows 'Output Program:' and line 22 shows 'Selamat, Anda Lulus!'. Line 23 is empty. <pre>1 2 Intermediate Code: 3 0 INT 0 5 4 1 LIT 0 75 5 2 STO 0 3 6 3 LOD 0 3 7 4 LIT 0 70 8 5 OPR 0 10 9 6 JPC 0 12 10 7 LIT 0 1 11 8 STO 0 4 12 9 LIT 0 0 13 10 OPR 1 14 14 11 JMP 0 16 15 12 LIT 0 0 16 13 STO 0 4 17 14 LIT 0 1 18 15 OPR 1 14 19 16 RET 20 21 Output Program: 22 Selamat, Anda Lulus! 23</pre>
6	Input <pre>ProgramNode(name: 'DivZero') ├── Declarations │ └── VarDecl('hasil') → tab_index:43, type:unknown, lev:0 ├── Block → block_index:42, lev:0 │ ├── Assign('hasil' := 10 div 0) → type:void │ │ ├── target 'hasil' → tab_index:43, type:integer │ │ └── value BinOp 'idiv' → type:integer │ │ ├── 10 → type:integer │ │ └── 0 → type:integer │ └── writeln(...) → predefined, tab_index:-1 │ └── arg 'hasil' → tab_index:43, type:integer</pre> Output Intermediate Code: 0 INT 0 4

	<pre> 1 LIT 0 10 2 LIT 0 0 3 OPR 0 5 4 STO 0 3 5 LOD 0 3 6 OPR 0 14 7 RET Output Program: [RUNTIME ERROR] [DivisionByZero] Pembagian atau modulus dengan nol tidak diperbolehkan.</pre>
	Screenshot Output
	 <pre> test > milestone-4 > test > output-6.txt 1 2 Intermediate Code: 3 0 INT 0 4 4 1 LIT 0 10 5 2 LIT 0 0 6 3 OPR 0 5 7 4 STO 0 3 8 5 LOD 0 3 9 6 OPR 0 14 10 7 RET 11 12 Output Program: 13 [RUNTIME ERROR] [DivisionByZero] Pembagian atau modulus dengan nol tidak diperbolehkan. 14</pre>

4. Kesimpulan Dan Saran

4.1 Kesimpulan

Pengembangan sistem kompilator "Arion Compiler" telah berada di tahapan tuntas yang ditandai oleh fungsionalnya fase analisis sintaks-semantik menjadi hasil nyata (eksekusi output). Konversi *Decorated AST* menggunakan mekanisme *Three-Address Code (TAC)* dan evaluasi pada Interpreter mesin *Virtual* berbasis *Stack* berhasil dieksekusi dengan baik. Kompilator ini telah dibekali dengan keamanan *interpreter* (seperti penanganan iterasi tak terhingga dan *out-of-bounds array*) sehingga beroperasi mandiri dan kuat terhadap program tak terduga.

4.2 Saran

Implementasi ICG saat ini berhasil di tingkat *depth-first search* dengan penanganan kontrol alur sederhana. Ke depannya, Interpreter dapat dilengkapi dengan sistem optimasi kompilasi

tambahan (seperti menghapus variabel mati/tidak bergunanya blok `jump` yang berdekatan) serta meluaskan kapabilitas fungsi-bawaan secara lebih kaya.

5. Lampiran

Github: <https://github.com/Rmadhian/DGN-Tubes-IF2224-2026>

Diagram:

https://drive.google.com/file/d/1Oyf-TSpWNtV-PG_HnQljXHczzvb1DiUa/view?usp=sharing

Tabel Kontribusi:

NIM	Kontribusi	Persentase
13523150	Mengimplementasi <code>icg_stmt.cpp</code> , <code>interpreter_security.cpp</code> , & <code>interpreter_flow.cpp</code>	33,33 %
13524117	-	0 %
13524120	Mengimplementasi <code>icg_decl.cpp</code> & <code>interpreter_core.cpp</code>	33,33 %
13524126	Mengimplementasi <code>icg_expr.cpp</code> & <code>interpreter_expr.cpp</code>	33,34%

6. Referensi

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, techniques, and tools* (2nd ed.). Pearson Education.

[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles.%20Techniques.%20and%20Tools%20(2006).pdf)